



KAMILIMU COHORT 10 • CLOUD ENGINEERING TRACK

Cloud Automation & DevOps

From "works on my machine" to "deploys on every commit" — pipelines, containers, and infrastructure as code



What we'll cover

01 The DevOps mindset

Why culture matters more than tools, shared ownership.

02 CI/CD pipelines

Continuous integration, delivery, deployment. The pipeline anatomy and modern deployment strategies.

03 Containers

Docker, registries, orchestration with ECS & EKS.

04 Infrastructure

Terraform vs CloudFormation

05 The tool landscape

The DevOps stack — Git, GitHub Actions, Docker, Terraform, Kubernetes.

06 Live demo

Walk through the carrental-project — a real deployable app.

PART 01

The DevOps Mindset

Culture, practices, tools — in that order



What DevOps actually is

Not a job title. Not a tool. Not a checklist. It's a way of building and running software.



Developers and operations sharing ownership of the whole lifecycle.

From writing code to running it in production. Same team, same goals, same incident channel.



Culture

Shared responsibility. No "throw it over the wall."
Blameless postmortems. Small autonomous teams.
Fast feedback loops.



Practices

CI/CD. Infrastructure as code. Automated tests.
Small frequent releases. Feature flags. Continuous
monitoring.



Tools

Git, GitHub Actions, Docker, Terraform, Kubernetes,
Prometheus. But: tools serve the culture, not the
other way around.

The wall they broke down

Before DevOps, dev teams and ops teams had opposing incentives. That's what needed to change.



BEFORE

Devs vs Ops

Two teams. Two incentive structures. One production incident.

- Devs measured on features shipped — push change fast
- Ops measured on uptime — resist change to keep stability
- Code "thrown over the wall" to ops at release time
- Ops fights fires devs caused; devs blame the environment
- Releases quarterly, painful, and everyone dreads them



NOW

One team, one lifecycle

Shared metrics, shared on-call, shared postmortems.

- "You build it, you run it" — devs on call for their code
- Deployment automated — no more manual release windows
- Every commit tested and shippable within minutes
- Feedback fast — errors surface in staging, not in customers
- Releases multiple times per day, boring, and reversible

PART 02

CI/CD Pipelines

From commit to production — automated



CI, CD, and Continuous Deployment — three related things

People say "CI/CD" and mean different things. Here's the real distinction.

CI

Continuous Integration

Every commit is merged to the main branch, then built and tested automatically. Broken builds are fixed immediately.

The question it answers

"Does the code still work?"

CD

Continuous Delivery

Main branch is ALWAYS in a deployable state. A human decides WHEN to release with the push of a button.

The question it answers

"Can we release right now?"

CD²

Continuous Deployment

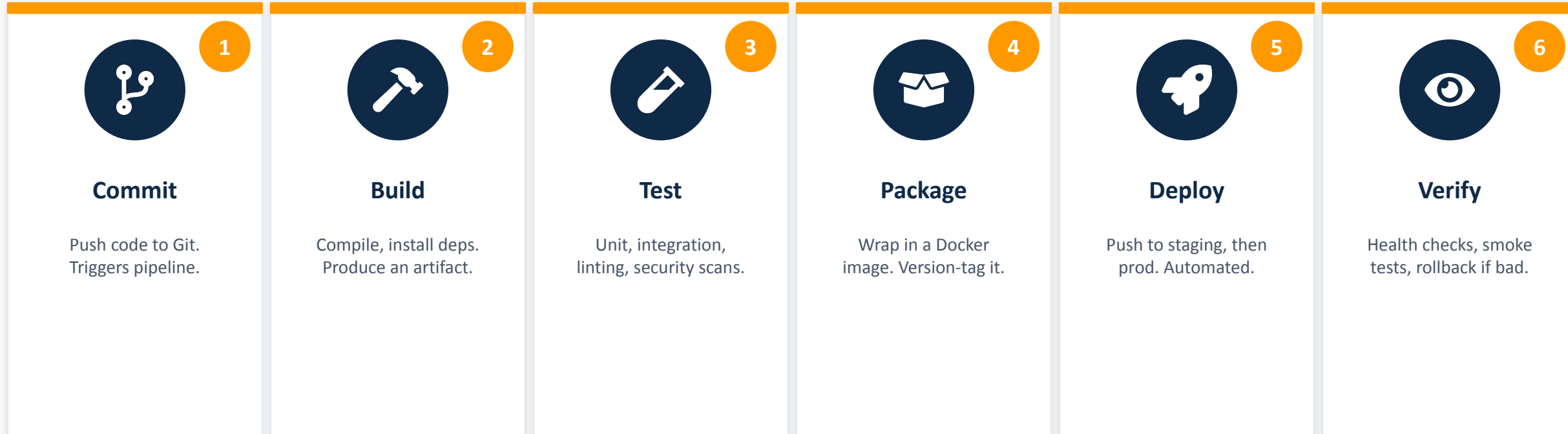
Every green commit ships to production automatically. No human release step. The pipeline is the release.

The question it answers

"Is every commit in production?"

Anatomy of a CI/CD pipeline

Every pipeline is a variation on this six-stage flow. The details change, the shape doesn't.



Fail fast — fail cheap

A test failure at stage 3 costs seconds. A production incident at stage 6 costs hours. Push every check as far LEFT in the pipeline as possible.

GitHub Actions — the pipeline that lives in your repo

YAML files in `.github/workflows/` tell GitHub what to run on every push, PR, or schedule. Free for public repos.

```
.github/workflows/deploy.yml
```

```
name: Build and Deploy
```

```
on: [push]
```

```
jobs:
```

```
  build:
```

```
    runs-on: ubuntu-latest
```

```
    steps:
```

- `- uses: actions/checkout@v4`
- `- run: docker build -t app .`
- `- run: docker push ecr.aws/app`

```
  deploy:
```

```
    needs: build
```

```
    runs-on: ubuntu-latest
```

```
    steps:
```

- `- run: aws ecs update-service ...`

Workflow

A YAML file. One file = one pipeline.

Job

A unit of work. Jobs run in parallel by default.

Step

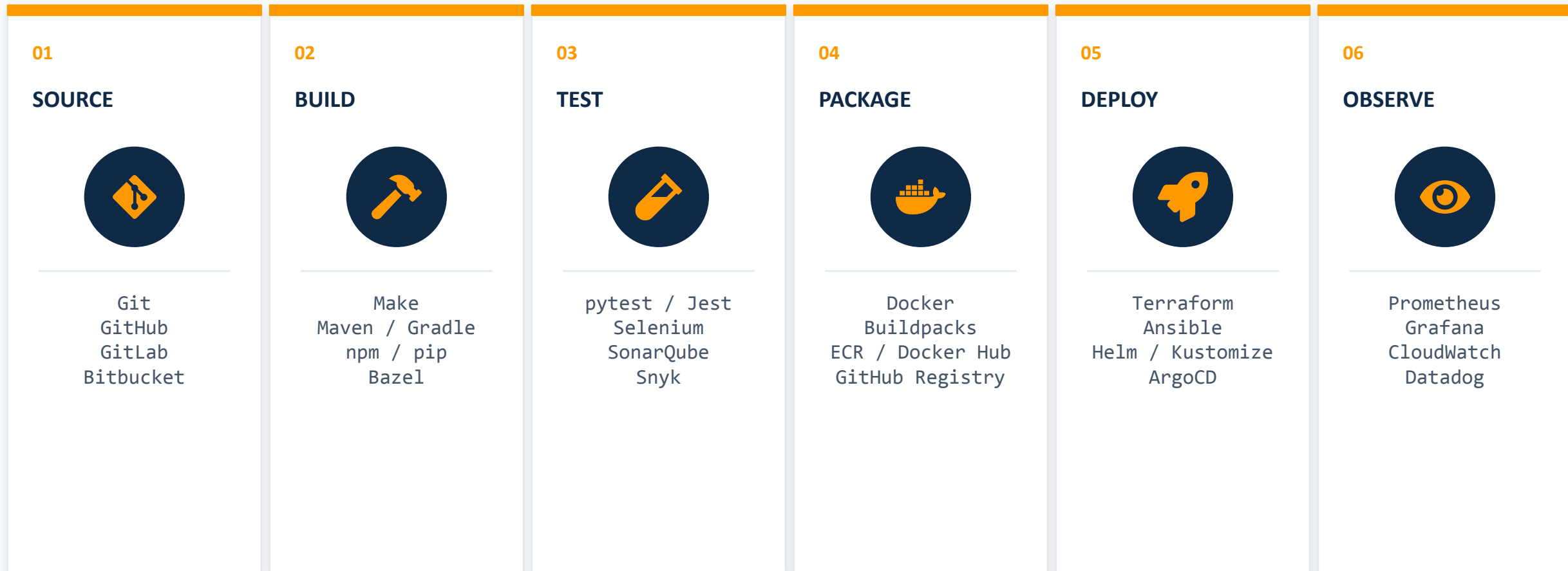
A shell command or a reusable action.

Runner

An ephemeral VM that executes the job. Ubuntu, macOS, Windows, or self-hosted.

The DevOps tool landscape

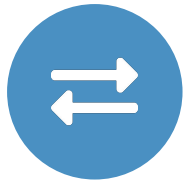
Six stages. A tool for each — sometimes several. Here's what you'll actually see in the wild.



Loops back to Source on every deploy

Deployment strategies — how you release without breaking things

"Just deploy to prod" is a career-ending strategy. Real teams use one of these three patterns.



Blue / Green

Two identical production environments. Deploy new version to the idle one. Test it. Switch traffic. Old env kept as instant rollback.

Upside

Instant rollback. Zero downtime.

Downside

Costs 2x infra during deploy. Database changes still tricky.



Canary

Route a small % of traffic (say 5%) to the new version. Watch metrics. Increase gradually if healthy. Roll back on error signals.

Upside

Catches regressions before all users hit them.

Downside

Needs good metrics and a smart load balancer.



Rolling

Replace instances one at a time. Take one down, deploy new version, add back to pool. Repeat until all instances are updated.

Upside

No extra infra. Simple to set up.

Downside

Both versions live during deploy — API compat matters.

PART 03

Containers

Packaging code so it runs anywhere



Docker in one slide

A container bundles an app + all its dependencies into a single artifact that runs the same anywhere. Solves "works on my machine" forever.



Dockerfile

The recipe

A plain text file describing how to build the image. Base image + copy files + install deps + define start command.



Image

The built artifact

A read-only template. Layered filesystem — each Dockerfile instruction becomes a cached layer. Immutable once built.



Container

The running instance

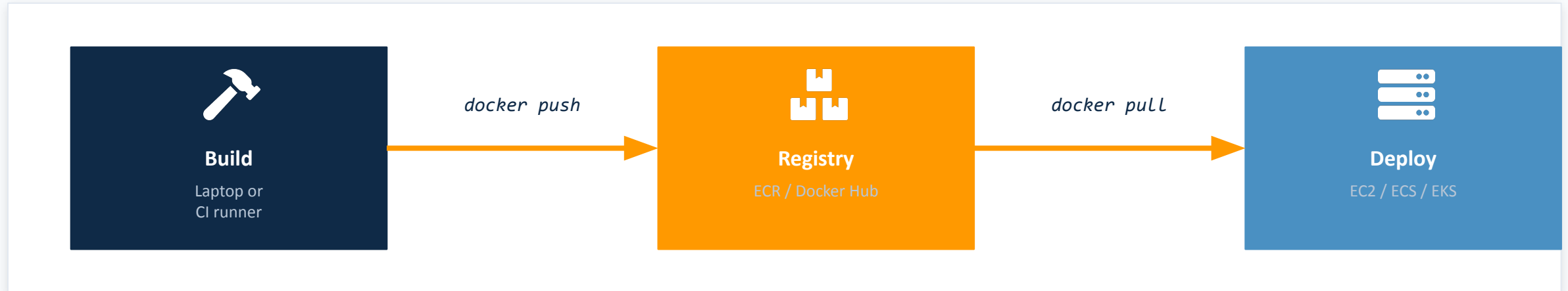
A running process launched from an image. Isolated CPU, memory, and network. Start, stop, delete. Cheap and disposable.

Dockerfile — a real example

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
COPY . .
RUN pip install -r requirements.txt
CMD ["python", "app.py"]
```

Registries — where images live

You build the image on your laptop or CI runner, then push to a registry so other machines can pull it.



Amazon ECR

AWS-native

Private by default. IAM-gated. Integrates with ECS / EKS. Pay per GB stored + transfer. First 500 MB free.

Docker Hub

Public option

Free for public images. Paid for private. Rate-limited on anonymous pulls — annoying in CI.

GitHub Registry

Ships with GitHub

Free with your repos. IAM tied to GitHub. Great when your CI is already GitHub Actions.

ECS vs EKS — running containers at scale on AWS

One container is easy. Fifty containers across ten servers with health checks and rolling deploys? That's orchestration.



ECS

Elastic Container Service

AWS-native orchestrator. Simpler mental model. Deeply integrated with AWS.

- Two launch types: EC2 (you manage nodes) or Fargate (serverless)
- Simple task-definition JSON — no CRDs, no Helm
- IAM tied directly to task roles
- AWS-only — no portability if you multi-cloud
- Best for: teams that just want containers running on AWS



EKS

Elastic Kubernetes Service

Managed Kubernetes control plane. Massive ecosystem. Portable across clouds.

- Industry-standard Kubernetes — huge community
- Runs the same workloads across AWS, GCP, on-prem
- Helm, ArgoCD, service mesh — deep ecosystem
- Steep learning curve. Operational complexity is real
- Best for: multi-cloud, complex apps, teams with K8s expertise

PART 04

Infrastructure

Code your infrastructure.



Managing infrastructure the way you manage code



Your VPC, your EC2s, your S3 buckets — all defined in code.

Version-controlled, PR-reviewed, diffable, reproducible. "Click-ops" — pointing and clicking in the AWS console — is the enemy.

Declarative

You describe WHAT you want. The tool figures out HOW.

- "I want one EC2 with these SGs."
- The tool creates, updates, or destroys to reach that state
- Idempotent — running twice gives the same result
- Tools: Terraform, CloudFormation, Pulumi

Imperative

You write step-by-step commands. Order and state are your problem.

- "Run this bash, then that bash, then..."
- You handle idempotency yourself — usually poorly
- Great for one-off setup or wiring together services
- Tools: bash, Ansible (partially), AWS CLI scripts

Terraform vs CloudFormation

The two big IaC tools you'll meet on AWS. Both declarative. Different trade-offs.

	Terraform	CloudFormation
Origin	HashiCorp — open source	AWS-native
Language	HCL (custom DSL)	JSON or YAML
Cloud support	AWS, GCP, Azure, 100+ providers	AWS only
Modules	Huge community registry	Nested stacks + AWS-provided templates
State	You manage state file (S3 + DynamoDB)	AWS manages state — invisible to you
Drift detection	Via plan / third-party tools	Built-in
Best when	Multi-cloud, community modules matter	Deep AWS-only, no state to manage

What we've covered



DevOps is culture first

Shared ownership, blameless, small teams. Tools serve the culture.



Ship safely

Blue/green, canary, rolling. Never just "deploy to prod."



CI vs CD vs Deploy

Merge & test. Ready to release. Auto-release.



Pipeline anatomy

Commit → Build → Test → Package → Deploy → Verify.



Containers

Dockerfile. Image. Container. Portable everywhere.



Orchestration

ECS for simple. EKS for portable & complex.



IaC over click-ops

Terraform / CloudFormation. PR-reviewed, versioned.

LIVE DEMO

Walkthrough — carrental-project

A real app, wired for real DevOps. Everything from today's session, applied end-to-end.



1

The repo

Clone. Walk the structure. Where's the app? Where's the pipeline?

2

The Dockerfile

How the app is packaged. Base image, dependencies, entrypoint.

3

The workflow

.github/workflows — the pipeline definition. Read it top to bottom.

4

The registry

Where the built image lives. Show a real tagged version in ECR.

5

The deploy target

Where it runs — EC2, ECS, or EKS. Show it live and healthy.

6

The rollback

Trigger a bad deploy. Roll back via the pipeline. Watch it recover.

WATCH FOR

The commit → live loop

How long from git push to running in prod? Sub-10-minute is the goal.

What's automated

Every step you don't see manually run is a step DevOps has eliminated.

What's still manual

The manual bits are the next things to automate. Note them for questions.

Repo: github.com/Topsideboss2/carrental-project



THANK YOU

Questions?

Bring your broken pipelines, your "my Dockerfile won't build" errors, your Terraform state horror stories.

